You've now separated the string conversion and the task size string conversion into their own methods.

Refactoring to Single-Assertion Specs

There's nothing to refactor in the code right now, but you have a series of tests that have a common setup and a common set of assertions. This duplication can be managed in RSpec in a few different ways—including letting it stay the way it is if you find the tests quite readable in their current structure.

Also, there's a style of coding that prefers to have specs that contain just a single assertion. By default, RSpec ends the spec when the first expectation fails. This means that if there are any expectations later in the spec, you don't know whether they are correct. Sometimes this can lead to the tests giving an inaccurate sense of the status of the code.

The single-assertion style looks a little different than the testing style you've seen so far. It often takes advantage of the lazy nature of RSpec's let to specify the common action and allows each test to specify an input to the action. Here's an example:

test_first/03/spec/workflows/creates_project_spec.rb

```
require "rails_helper"

RSpec.describe CreatesProject do

  let(:creator) { CreatesProject.new(
    name: "Project Runway", task_string: task_string) }

  describe "initialization" do
    let(:task_string) { "" }
    it "creates a project given a name" do
      creator.build
      expect(creator.project.name).to eq("Project Runway")
    end
  end

  describe "task string parsing" do
    let(:tasks) { creator.convert_string_to_tasks }

    describe "with an empty string" do
      let(:task_string) { "" }
      specify { expect(tasks).to be_empty }
    end

    describe "with a single string" do
      let(:task_string) { "Start Things" }
      specify { expect(tasks.size).to eq(1) }
      specify { expect(tasks.first).to have_attributes(
        title: "Start Things", size: 1) }
    end
```